

Algoritmos Computacionais e Estrutura de Dados em Python

– Cap-06 – Lista (Vetor)

Lista (Vetores)

- **Exemplo 1:**

Faça um programa que leia matrícula e nota de 5 alunos e em seguida imprima as matrículas dos alunos cuja nota seja maior ou igual a média das 5 notas.

Lista (Vetores)

```
nome1=input('\n\nEntre com o nome1: ')
nota1=float(input('Entre com a nota1: '))

nome2=input('Entre com o nome2: ')
nota2=float(input('Entre com a nota2: '))

nome3=input('Entre com o nome3: ')
nota3=float(input('Entre com a nota3: '))

nome4=input('Entre com o nome4: ')
nota4=float(input('Entre com a nota4: '))

nome5=input('Entre com o nome5: ')
nota5=float(input('Entre com a nota5: '))

media = (nota1 + nota2 + nota3 + nota4 + nota5)/5
```

```
print("\nA média alunos é : ",media)

if (nota1 > media):
    print(f'{nome1} possui nota maior que a média ')

if (nota2 > media):
    print(f'{nome2} possui nota maior que a média ')

if (nota3 > media):
    print(f'{nome3} possui nota maior que a média ')

if (nota4 > media):
    print(f'{nome4} possui nota maior que a média ')

if (nota5 > media):
    print(f'{nome5} possui nota maior que a média ')
```

Lista (Vetores)

E se fossem 100 alunos ?

- É possível definir variáveis que guardam mais de um valor de um mesmo tipo
- Essas variáveis são conhecidas como variáveis indexáveis ou arranjos (array)
- Em Python essas variáveis são chamadas de listas.

Vetor

- **Variável composta unidimensional**
 - Contém espaço para diversos valores
 - É acessado via um índice
- A ideia de vetor é comum na matemática, com o nome de variável subscrita

Exemplo: $x_1, x_2, \dots, x_n$

Vetor

- **O que vimos até agora são variáveis com somente um valor**

Exemplo: $y = 123$

- **No caso de vetores, uma variável guarda ao mesmo tempo múltiplos valores**
 - Exemplo: $x_1 = 123$, $x_2 = 456$, ...
 - $X = [123, 456, \dots]$

Listas

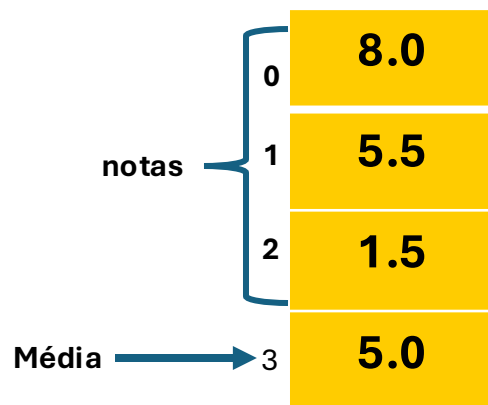
- **Em outras linguagens de programação, listas são chamadas de vetores e possuem restrições que Python não tem:**
 - Em Python, os valores de uma lista podem ser de **qualquer tipo**
 - Em outras linguagens, os valores precisam ser do **mesmo tipo**
- **Em Python**
 - Conjunto = ['A', 1,2,'Casa', 2.3]
 - Exemplo: x1 = 123, x2 = 456, ...
 - Notas = [10, 5, 6.7, 2, 7.5,...]

Listas

- Para acessar (ler ou escrever) uma posição do vetor, basta informar a posição entre colchetes

```
notas = [8.0, 5.5, 1.5]  
média = (notas[0] + notas[1] + notas[2]) / 3
```

Pode-se iterar por todos os seus valores usando um comando for



```
notas_média = [8.0, 5.5, 1.5, 0]  
média = (notas_média[0] + notas_média[1] + notas_média[2]) / 3  
notas_média[3] = média  
print('\n\nmédia : ',média,'\n')  
  
for i in range(4):  
    print(notas_média[i], end = ' ')
```

Como Inserir Elementos na Lista

- **Inicialmente cria-se uma lista vazia**

Ex.: `notas = []`

- **Depois adiciona-se valores na lista usando `append`**

Ex.:

```
n = float(input('Digite a nota do primeiro aluno: '))  
notas.append(n)
```

- Em Python, o **`append()`** é uma função que adiciona itens em uma lista pelo fim da lista.

Como Inserir Elementos na Lista - Append

```
notas = []
soma_notas = 0
for i in range(3): # é igual a range(0,3,1)
    notas.append(float(input(f'Entre com as notas[{i + 1}]: ')))
    soma_notas += notas[i]

notas.append(soma_notas / 3)

for i in range(3):
    print('\nNota ', i + 1, ' a : ', notas[i])

print('\n\nmédia : ', notas[3])
```

Como Inserir Elementos na Lista - Insert

Outra forma de incluir elementos em uma lista é utilizando o método **insert()**. A diferença desse método para o `append()` é que você especifica em qual índice deseja incluir o novo elemento. Observe a sintaxe básica:

<lista>.insert(<índice>, <elemento>)

```
atrizes = ["Paolla de Oliveira"]  
atrizes.append("Sheron Menezes")  
atrizes.insert(1, "Raquel Nunes")  
print(atrizes)
```

Ao fim da execução da linha 2, a lista **atrizes** estará da seguinte forma:

```
atrizes = ["Paolla de Oliveira", "Sheron Menezes"]
```

Métodos shuffle() e choice()

```
import random
atrizes = ["Adriana Prado", "Bárbara Borges", "Camila Queiroz", "Danielle Winitz", "Fernanda
Paes Leme", "Helena Ranaldi", "Paolla de Oliveira", "Raquel Nunes", "Viola Dias"]
```

```
random.shuffle(atrizes) #Embaralha a lista
print(f"Lista embaralhada: {atrizes}")
sorteada = random.choice(atrizes) #Sorteia aleatoriamente
print(f"Atriz sorteada: {sorteada}")
```

```
<lista>.sort(reverse = [False | True])
```

REMOVENDO ELEMENTOS

Para remover elementos de uma lista, existem os métodos **remove()** e **pop()** e a função **del**. As sintaxes básicas de cada uma dessas maneiras de remover elementos de uma lista são mostradas a seguir:

```
<lista>.remove(<elemento>) atrizes = ["Adriana", "Adriana", "Camila", "Danielle",  
<lista>.pop(<índice>)      "Fernanda", "Helena", "Paolla", "Raquel", "Viola"]  
del <lista>[<índice>]      atrizes.remove("Adriana")  
                           print(atrizes)  
                           atrizes.pop(1)  
                           print(atrizes)  
                           atrizes.pop()  
                           print(atrizes)  
                           del atrizes[1]  
                           print(atrizes)  
                           del atrizes #Destroi a variável atrizes
```

Método *Clear()* e *List()*

Por fim, se desejarmos esvaziar uma lista, podemos utilizar o método `clear()`, cuja sintaxe básica é a seguinte: **<lista>.clear()**

list(<lista>)

Onde `<lista>` é a lista que está sendo clonada. Como resultado, o valor retornado pode ser atribuído a uma variável, como no exemplo abaixo:

```
atrizes = ["Adriana", "Adriana", "Camila", "Danielle", "Fernanda", "Helena", "Paolla",  
"Raquel", "Viola"]
```

```
copia = list(atrizes)  
print(atrizes)  
print(copia)
```

Método Count() e Enumerate()

<lista>.count(<elemento>)

```
atrizes = ["Adriana", "Adriana", "Camila", "Danielle", "Fernanda", "Helena",  
          "Paolla", "Raquel", "Viola"]
```

```
ocorrencias = atrizes.count("Adriana")  
print(f"O elemento Adriana se repete {ocorrencias} vezes.")
```

```
atrizes = ["Adriana", "Bárbara", "Camila", "Danielle", "Fernanda"]  
for indice, nome in enumerate(atrizes):  
    print(f"{nome} está armazenado no índice {indice}")
```

Uma maneira diferente de gerar e manipular Listas

Existe uma maneira de se criar listas menos verbosa, ou seja, com menos código e mais compreensível. No mundo Python, isso se chama **List Comprehensions**, cuja sintaxe é:

novaLista = [expressao for item in lista [condicao]]

Onde **novaLista** é a nova lista gerada pela operação, **expressão** corresponde a expressão que deve gerar o valor de um elemento na lista, **item** é o elemento iterado, lista o conjunto de elementos que devem ser iterados e **condição** um teste opcional que pode ser usado para a geração de valores.

List Comprehensions

Em computação, *List Comprehension* é uma construção equivalente as notações matemáticas do tipo: $S=\{x^2 \forall x \text{ em } \mathbb{N}, x \geq 20\}$

Ou seja, S é o conjunto formado por x ao quadrado para todo x no conjunto dos números naturais, se x for maior ou igual a 20.

Sintaxe: **lista = [<expressão> for <referência> in <sequência> if <condição>]**



```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
# Eleva os ímpares ao quadrado [1,9,25,49,81,121]
print( [ x**2 for x in nums if x % 2 == 1 ] )
```

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
for x in nums:
    if (x % 2 == 1):
        print(x ** 2, end= " ")
```



```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
print()
for x in range(len(nums)):
    if (nums[x] % 2 == 1):
        print(nums[x] ** 2, end= " ")
```

List Comprehensions

```
lista1 = [x ** 2 for x in range(10)]
```

```
print(lista1) ← # 0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
lista2 = []
```

```
lista2 = [x for x in range(1,20) if x % 2 == 0]
```

```
print(lista2) ← # [2, 4, 6, 8, 10, 12, 14, 16, 18]
```

```
lista3 = [i for i in "HACKATHON" if i in ["A","E","I","O","U"]]
```

```
print(lista3) ← # ['A', 'A', 'O']
```

```
lista4 = [1,2,3]
```

```
lista4 = [i**3 for i in lista4]
```

```
print(lista4) ← # [1, 8, 27]
```

***HACKATHON** é uma maratona de programação na qual programadores, designers e outros profissionais de programação se reúnem a fim de construir soluções inovadoras para um problema apresentado.

Listas

- **Exemplo2:**

Faça um programa que leia matrícula e nota de 10 alunos e em seguida imprima as matrículas dos alunos cuja nota seja maior ou igual a média das 10 notas

Se necessário crie duas listas (vetores) com 10 posições em cada lista.

- Uma lista para nomes
- Uma lista para notas

Listas

```
tot_alu = 5
nota = []
nome = []
media = 0
for i in range(tot_alu):
    nome.append(input(f'Entre com o nome[{i+1}]: '))
    nota.append(float(input(f'Entre com a nota[{i+1}]: ')))
    media += nota[i] # Igual a media = media + nota[i]

media/=tot_alu # media = media / tot_alu

print("\nA média alunos é : ",media)

print(f'\n\nAlunos que possuem nota maior
que a média das {tot_alu} notas:')

for i in range(tot_alu):
    if (nota[i] >= media):
        print(nome[i])
```

Índices para Acesso aos Elementos da Lista

- Python permite acesso à lista em ordem crescente ou decrescente de posição
 - Primeira posição é 0
 - Última posição é -1

```
Prompt de Comando - python
>>>
>>> c = [-45, 6, 0, 72, 1543]
>>> c[3]
72
>>> c[-2]
72
>>> c[0] == c[-5]
True
>>>
```

C[0]	-45	C[-5]
C[1]	6	C[-4]
C[2]	0	C[-3]
C[3]	72	C[-2]
C[4]	1543	C[-1]

Função len

- Retorna a quantidade de elementos que a lista possui

```
Prompt de Comando - python
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Almir>python
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug  6 2024, 20:45:27) [MSC v.1940
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> numeros = [3,1,6,7,10,22,4]
>>> len(numeros)
7
>>>
```

Concatenação entre listas

```
Prompt de Comando - python
>>>
>>>
>>>
>>>
>>> lista = [1,2,3]
>>> lista = lista + [4]
>>> lista = lista + [4,5,6]
>>> lista
[1, 2, 3, 4, 4, 5, 6]
>>>
```

Concatenação entre listas

- Exemplo 3:
- Faça um programa que leia um número inteiro n, e retorne uma lista com todos os números pares entre 2 e um número n, inclusive.

```
lista = []  
n=int(input('\n\nEntre com um número inteiro : '))  
for i in range(2,n+1,2):  
    lista += [i];           # lista = lista + [i]  
print(lista)
```

Entre com o número inteiro : 10

lista = [i] + lista

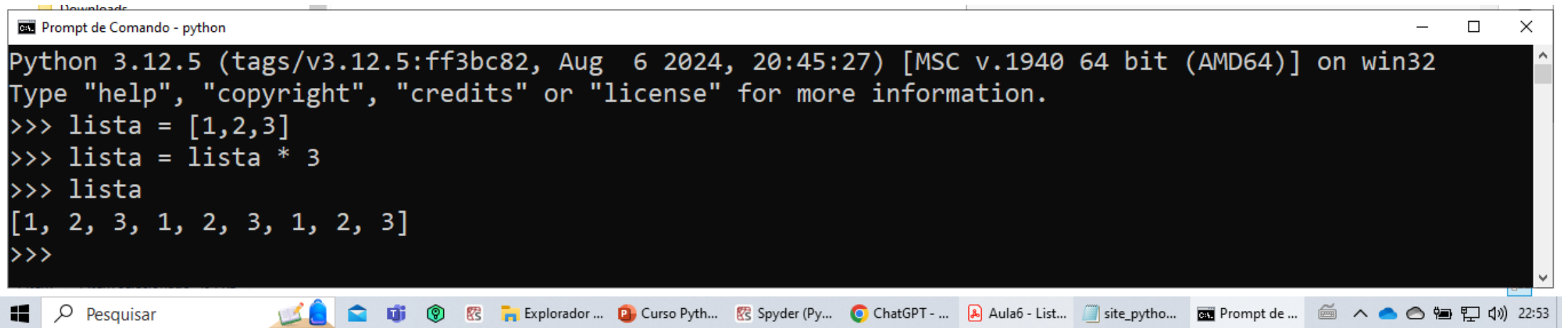
2 4 6 8 10

Concatenação entre listas

1. `Range(2, n+1, 2)` gera apenas os **pares crescentes** de 2 até n.
 - Exemplo: se $n=10$, o i será: 2, 4, 6, 8, 10.
2. A linha `lista = [i] + lista` cria uma nova lista colocando i **na frente** da lista atual.
 - Isso significa que cada novo número é adicionado no **início** da lista (e não no final).
3. Assim, mesmo que i vá aumentando (2, 4, 6...), cada valor novo vai ficando **antes** dos anteriores.
 - Exemplo com $n=10$:
 - Iteração 1: `lista = [2]`
 - Iteração 2: `lista = [4] + [2] = [4, 2]`
 - Iteração 3: `lista = [6] + [4, 2] = [6, 4, 2]`
 - Iteração 4: `lista = [8] + [6, 4, 2] = [8, 6, 4, 2]`
 - Iteração 5: `lista = [10] + [8, 6, 4, 2] = [10, 8, 6, 4, 2]`
 - ➔ Resultado final: [10, 8, 6, 4, 2] (ordem decrescente)

“Multiplicação” de Listas

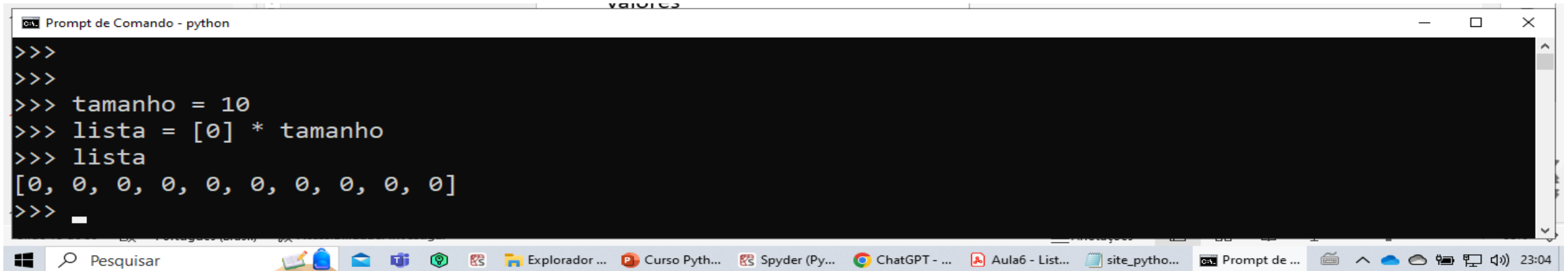
- O operador “*” repete n vezes os elementos que já estão na lista
- A operação $\text{lista} * n$ equivale a $\text{lista} + \text{lista} + \dots + \text{lista}$ (n vezes)



```
Prompt de Comando - python
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> lista = [1,2,3]
>>> lista = lista * 3
>>> lista
[1, 2, 3, 1, 2, 3, 1, 2, 3]
>>>
```

Inicialização de Listas com Zero

- Em diversas situações onde **já sabemos de antemão qual será o tamanho de uma lista** de inteiros, é útil inicializar a lista com o valor 0
- Isso evita que precisemos usar o **append** para adicionar valores



```
Prompt de Comando - python
>>>
>>>
>>> tamanho = 10
>>> lista = [0] * tamanho
>>> lista
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
>>> _
```

Inicialização de Listas com Zero

Exemplo5:

- Faça um programa que crie uma lista com 5 notas e em seguida calcule a média, **sem utilizar append**

```
tam = 5
lista = [0]*tam
media = 0
for i in range(tam):
    lista[i] = int(input(f'Entre com lista[{i}] : '))
    media += lista[i]    # igual a → media = media + lista[i]
media/=tam             # igual a → media = media / tam
print('\n\n',lista)
print(f'\n\n{media}')
```

Teste de Pertinência – Elemento *in* Lista

Elemento *in* lista

- Verifica se um determinado valor pertence ou não pertence a uma lista
- Retornar True caso o valor pertença à lista, e False caso contrário

Define a lista de valores

lista = [10, 20, 30, 40, 50]

Solicita o valor a ser verificado

valor = int(input("Digite um valor para verificar se ele pertence à lista: "))

Verifica se o valor está na lista

pertence = valor in lista

Imprime True se o valor pertence à lista, False caso contrário

print(pertence)

Teste de Pertinência – Elemento *in* Lista

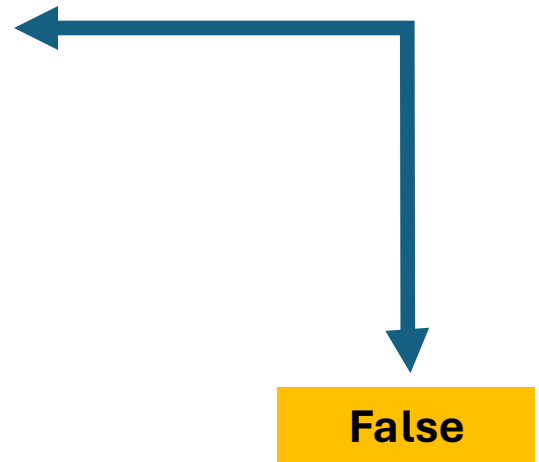
- Exemplo6:

Verificar se o número 7
pertence a lista abaixo.

```
lista = [1,2,3,4,5]
```

```
resultado = 7 in lista
```

```
print(resultado)
```



False

- Exemplo7:

Verificar se o número 7 pertence
a lista abaixo, sem usar *in*.

```
lista = [1,2,3,4,5]
```

```
#[1,2,3,7,4,5]
```

```
valor = 7
```

```
resultado = False
```

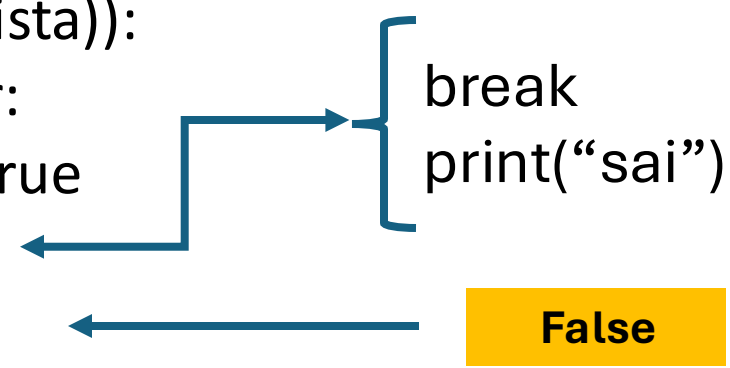
```
for i in range(len(lista)):
```

```
    if lista[i]==valor:
```

```
        resultado = True
```

```
        break  
        print("sai")
```

```
print(resultado)
```



False

Conhecimento útil: split() retorna um vetor

- Utilizado na função input()
 - Só permite criar uma lista de cadeia de caracteres!!!!
 - A função int() não pode ser utilizada para converter uma lista de cadeia de caracteres em uma lista de inteiros!!!!!!!!!!

Ex.:

```
lista = input('Entre com valores separados por espaços: ').split()  
print(lista)
```

- Split é uma função que retira os espaços entre os elementos da lista

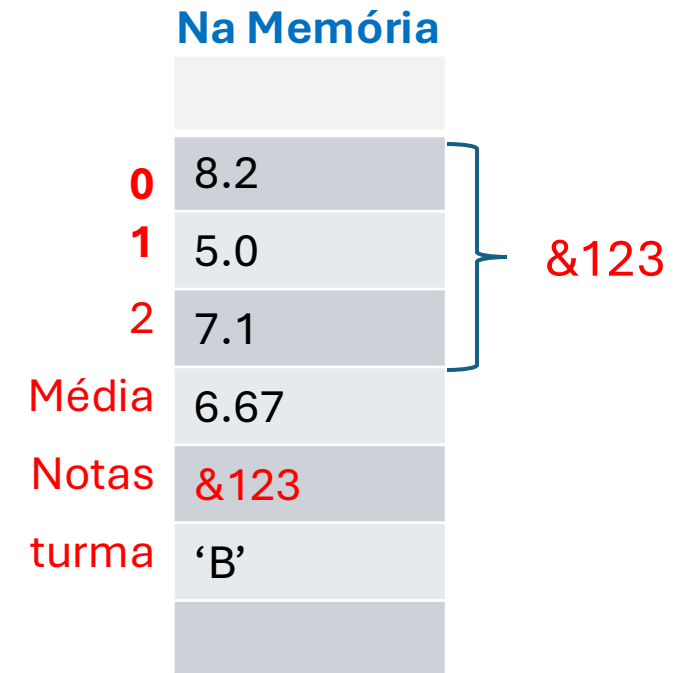
Entre com valores
separados por espaços: 1 2
3 4 5 6
['1', '2', '3', '4', '5', '6']

Representação de Listas em Memória

- O valor de uma variável de lista na verdade é um endereço de memória

Em Python

```
notas = [8.2, 5.0, 7.1]
turma = 'B'
media = 0
tot_notas = 0
for i in range (len(notas)):
    tot_notas = tot_notas + notas[i]
media = media + tot_notas / len(notas)
print(f'{media:.2f}')
```

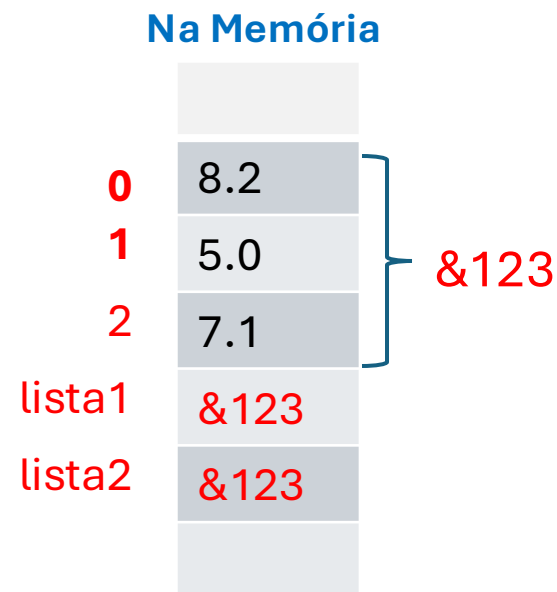


Cópia de Listas

- Ao **copiar uma lista para outra**, o que é feito é **copiar** o valor do **endereço de memória**
- **Ambas** passam a apontar para o **mesmo endereço**, portanto **o que for modificado em uma lista também será modificado na outra**

Em Python

```
>>> lista1 = [1,2,3]
>>> lista2 = lista1
```

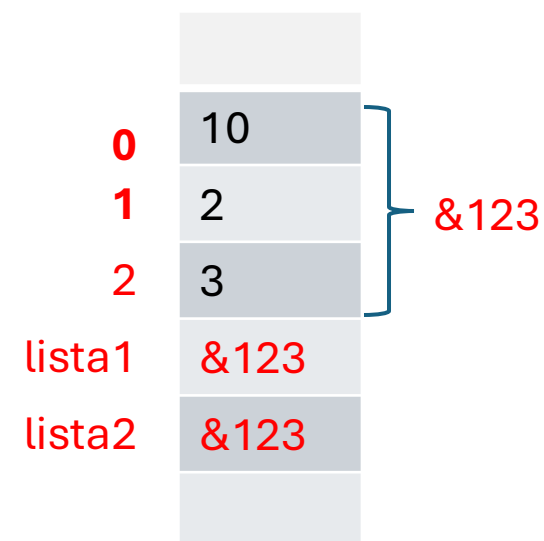


Cópia de Listas

Em Python

```
Prompt de Comando - python
>>>
>>> lista1 = [1,2,3]
>>> lista2 = lista1
>>> lista1[0] = 10
>>> lista1
[10, 2, 3]
>>> lista2
[10, 2, 3]
>>> _
```

Na Memória

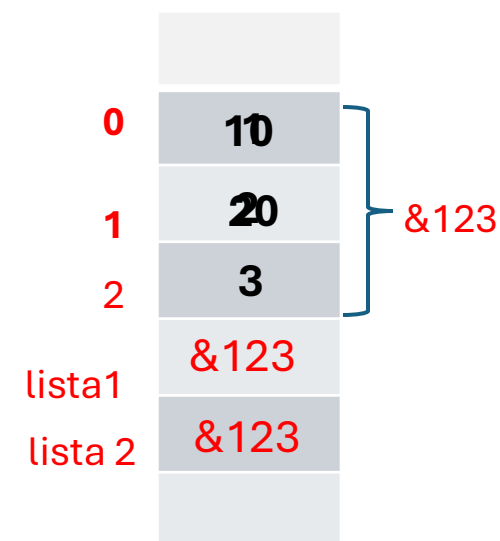


Cópia de Listas

Em Python

```
Prompt de Comando - python
>>>
>>> lista1 = [1,2,3]
>>> lista2 = lista1
>>> lista1[0] = 10
>>> lista1
[10, 2, 3]
>>> lista2
[10, 2, 3]
>>> lista2
[10, 2, 3]
>>> lista2[1] = 20
>>> lista2
[10, 20, 3]
>>> lista1
[10, 20, 3]
>>>
```

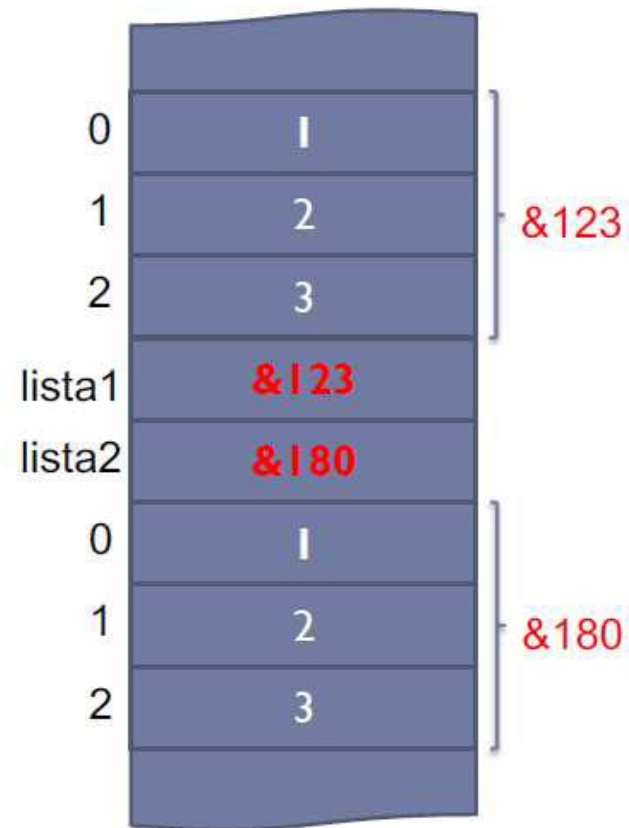
Na Memória



Cópia de Listas

Exemplo

```
>>> lista1 = [1, 2, 3]
>>> lista2 = []
>>> for i in range(len(lista1)):
...     lista2.append(lista1[i])
... 
```



Cópia de Listas

```
C:\> Selecionar Prompt de Comando - python

>>>
>>> lista2 = []
>>> lista1 = [1,2,3,4,5]
>>> for i in range(len(lista1)):
...     lista2.append(lista1[i])
...
>>>
>>> lista2[0] = 10
>>> lista1
[1, 2, 3, 4, 5]
>>> lista2
[10, 2, 3, 4, 5]
>>> lista1[3] = 20
>>> lista2
[10, 2, 3, 4, 5]
```

Cópia de Listas – Outra maneira

Prompt de Comando - python

```
>>>
>>> lista1=[1,2,3,4,5,6]
>>> lista2=lista1
>>> lista3=lista1[:]
>>> print(id(lista1))
1530763301696
>>> print(id(lista2))
1530763301696
>>> print(id(lista3))
1530759459968
>>>
```

lista1 e lista2
apontam para o
mesmo endereço

Lista3 aponta
para o outro
endereço

Resposta
1530763301696
1530763301696
1530759459968

**Lista1 e lista2
possuem os
mesmos
endereços
Lista3 tem outro
endereço de
memória**

Como apagar um elemento de uma Lista?

- Função **del()**

```
Prompt de Comando - python
>>>
>>> lista = ['a','b','c','d','e','f']
>>> lista
['a', 'b', 'c', 'd', 'e', 'f']
>>> del (lista[2])
>>> lista
['a', 'b', 'd', 'e', 'f']
>>> lista = ['a', 'b', 'c', 'd', 'e', 'f']
>>> del (lista[1:5])
>>> lista
['a', 'f']
>>>
```

Como contar o número de ocorrências em uma Lista?

- `count()`

```
Prompt de Comando - python
>>> lista = [1,2,1,2,1,2,3,4,5]
>>> print(lista.count(1))
3
>>>
```

Resposta
3

Menor e Maior Elemento de uma Lista

- `min()` → retorna o menor elemento da lista
- `max()` → retorna o maior elemento da lista

```
Python Prompt de Comando - python
>>>
>>> lista = [1, 2, 3, 4, 5, 6]
>>> print(min(lista))
1
>>> print(max(lista))
6
>>>
```

Resposta

1

6

Retorna o índice de um elemento no vetor

- Pode-se encontrar a posição de um elemento em um vetor utilizando o método `index()`.

```
Prompt de Comando - python
>>>
>>>
>>>
>>> vetor = [5, 3, 10, 1, 98]
>>> print(vetor.index(10))
2
>>>
```

10 está na posição 2 do vetor (a contagem começa da posição 0)

Resposta
2

Função *sorted()*

- Ordena uma lista

```
Prompt de Comando - python

>>>
>>> a = [1, 2, -7, 89, 43, 14]
>>> b = sorted(a)
>>> print(a)
[1, 2, -7, 89, 43, 14]
>>> print(b)
[-7, 1, 2, 14, 43, 89]
>>>
```

[1, 2, -7, 89, 43, 14]

[-7, 1, 2, 14, 43, 89]

Função Lambda →

```
amp = lambda x, y, z: (x ** 2 + y ** 2 + z ** 2) ** .5
print (amp(1, 1, 1))    # 1.7320508075688772
print (amp(3, 4, 5))    # 7.0710678118654755
```

Mapeamento

- O mapeamento consiste em aplicar uma função a todos os itens de uma sequência, gerando outra lista contendo os resultados e com o mesmo tamanho da lista inicial.

No Python, o mapeamento é implementado pela função `map()`. Exemplos:

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
# log na base 10
```

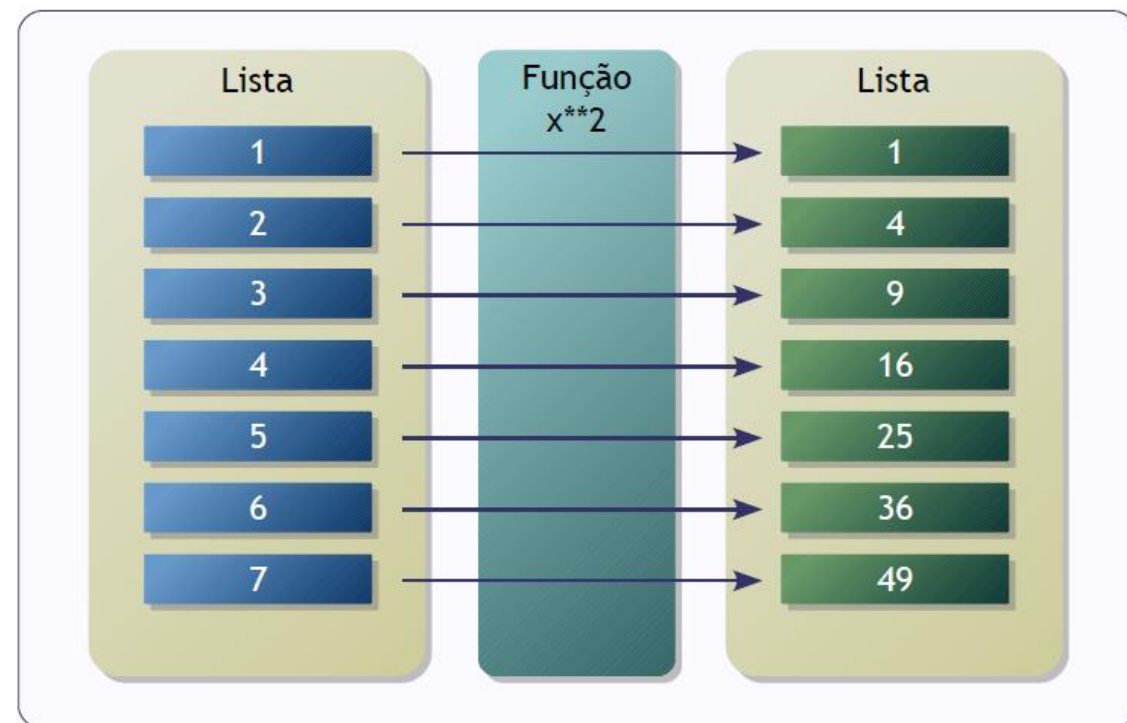
```
from math import log10
```

```
print map(log10, nums)
```

```
# Dividindo por 3
```

```
print map(lambda x: x / 3, nums)
```

A função `map()` sempre retorna uma lista.



1. `lambda x: x / 3`

É uma **função anônima** (sem nome).

- Recebe um valor `x`
- Retorna `x / 3`

2. `map(...)`

A função `map` aplica uma função a **cada elemento de uma lista (ou iterável)**.

`map(lambda x: x / 3, nums)` Significa: 🙌 “Pegue cada elemento da lista `nums` e divida por 3”

```
nums = [3, 6, 9, 12]
resultado = map(lambda x: x / 3, nums)
print(list(resultado)) # [1.0, 2.0, 3.0, 4.0]
```

O `map` não retorna uma lista diretamente, mas um **objeto iterável**.

Por isso, geralmente usamos:

```
list(map(...))
```

Usando **list comprehension** fica mais simples e mais “pythônico”

```
resultado = [x / 3 for x in nums]
```

✅ **Forma equivalente com `for`**

```
resultado = []
for x in nums:
    resultado.append(x / 3)
```

Comparação rápida

- 👉 Em provas e na prática, **list comprehension é mais usada** porque:
- é mais legível
 - mais direta
 - não precisa de lambda

Forma

`map`

`list comprehension`

Código

```
list(map(lambda x: x / 3, nums))
```

```
[x / 3 for x in nums]
```

Vetor - Matrizes

Exemplo com try - except

	0	1	2	3	4
notas	10.7	7,0	9,0	5.5	6.0

```
while True:
    try:
        n1 = int(input("\nDigite o
            primeiro número: "))
        n2 = int(input("Digite o
            segundo número: "))

        resultado = n1 / n2
        print("\nO resultado da
            divisão é: ",
            resultado)

        break

    except ValueError:
        print("Erro de Entrada
            inválida")

    except ZeroDivisionError:
        print("\nErro de Divisão por
            zero")
```

```
tot = 5
vet = []
for i in range(tot):
    num = int(input(f"\nDigite o {i+1}º valor do vetor : "))
    vet.append(num)

numpesq = int(input("\nDigite um número : "))
achei = False

for i in range(tot):
    if (numpesq == vet[i]):
        achei = True
        indice = i
        break

if (achei == True):
    print('\nA posição de ', numpesq,'no vetor : ',indice)
else:
    print('\nA posição de ', numpesq,'no vetor : -1, logo
        não existe')
```

```
vetor = []
tot_num = int(input("Digite o total de números no vetor :"))

# Preenche o vetor com 5 valores fornecidos pelo usuário
for i in range(tot_num):
    valor = int(input(f"Digite o {i+1}º valor: "))
    vetor.append(valor)

# Solicita o valor x a ser pesquisado no vetor
x = int(input("Digite o valor a ser pesquisado no vetor: "))

# Tenta encontrar a posição do valor x no vetor usando
    index()

try:
    posicao = vetor.index(x)
    print(f"O valor {x} está na posição {posicao} do vetor.")
except ValueError:
    print(-1)
```

Exercícios

1. Faça um programa que leia dois vetores de 3 posições, que representam forças sobre um ponto no espaço 3D, e escreva a força resultante. Dica: força resultante é obtida pela soma dos valores das coordenadas correspondentes nos dois vetores: $(x_1 + x_2)$, $(y_1 + y_2)$, $(z_1 + z_2)$
2. Faça um programa que preencha por leitura um vetor de 10 posições, e conta quantos valores diferentes existem no vetor.
3. Faça um programa que preencha por leitura um vetor de 5 posições, e informe a posição em que um valor x (lido do teclado) está no vetor. Caso o valor x não seja encontrado, o programa deve imprimir o valor -1. Faça a questão de duas formas: não utilizando o método `index()` e utilizando o método `index()`.

Exercícios

4. Um dado é lançado 50 vezes, e o valor correspondente é armazenado em um vetor. Faça um programa que determine o percentual de ocorrências de face 6 do dado dentre esses 50 lançamentos. Obs: utilize a função `randint()` para gerar os 50 lançamentos do dados. `randint(1,6)` → gera números inteiros aleatórios de 1 a 6. Insira no seu programa: `from random import randint`

Biblioteca Random

Função randint()

Uma das principais funções do módulo random em Python é a função randint(). Essa função permite gerar números inteiros aleatórios dentro de um intervalo específico. Para usar a função randint(), você precisa importar o módulo random no seu código.

Aqui está um exemplo de como usar a função randint() para gerar um número aleatório entre 1 e 10 em Python:

```
import random
```

```
numero_aleatorio = random.randint(1, 10)  
print(numero_aleatorio)
```

Neste exemplo, a função randint() gera um número inteiro aleatório entre 1 e 10, inclusive. Em seguida, o número aleatório é armazenado na variável “numero_aleatorio” e é impresso na tela.

1 - Gerar um número aleatório entre dois valores específicos:

```
import random
```

```
numero_aleatorio = random.randint(10, 20)  
print(numero_aleatorio)
```

Neste exemplo, a função randint() gera um número aleatório entre 10 e 20, inclusive.

Biblioteca Random

2 - Simular o lançamento de um dado:

```
import random
```

```
número_aleatório = random.randint(1, 6)  
print(número_aleatório)
```

Neste exemplo, a função randint() é usada para simular o lançamento de um dado de seis lados. O resultado será um número inteiro aleatório entre 1 e 6.

3 - Selecionar um elemento aleatório de uma lista:

```
import random
```

```
lista = ['maçã', 'banana', 'laranja', 'morango']  
elemento_aleatório = random.choice(lista)  
print(elemento_aleatório)
```

Neste exemplo, a função choice() é usada para selecionar um elemento aleatório da lista dada. O resultado será um elemento aleatório da lista, como “banana” ou “laranja”.

Exercícios

5. Faça um programa que leia um vetor vet de 20 números inteiros. O programa deve gerar, a partir do vetor lido, um outro vetor chamado “**pos**” que contenha apenas os valores inteiros positivos de vet. A partir do vetor pos, deve ser gerado um outro vetor chamado “**semdup**” que contenha apenas uma ocorrência de cada valor de pos.
6. Dado um vetor de 100 elementos, determine o maior e o menor elemento do vetor. (Utilize as funções min() e max())
7. Dado um vetor de 100 elementos, determine o maior e o menor elemento do vetor. (Não utilize as funções min() e max())

Exercícios

8. Dado um vetor de 100 elementos, determine a diferença entre a soma dos elementos de índice par e a soma dos elementos de índice ímpar.
9. Dado um vetor com 20 elementos, gerar outro vetor que contenha somente números múltiplos de 3 encontrados no primeiro vetor.
10. Existem 20 candidatos à presidência, com códigos que variam de 1 até 20. Codificou-se 21 para votos brancos e 22 para votos nulos. Cada voto vem em um cartão (contendo só um voto) e o último cartão vem com o número 0 (zero). Faça um programa que auxilie a operação dos votos, imprimindo a quantidade de votos de cada candidato, o número de votos em branco, o número de votos nulos e o total de votantes. Imprima também o candidato que venceu a eleição e o número de votos do vencedor.